

Software Engineering
Unit - IV

Compiled by
M S Anand

Department of Computer Science

Text Book(s):

1. "Software Engineering: A Practitioner's approach – 9th edition, Roger Pressman, Bruce Maxim
2. "Software Testing – Principles and Practices", Srinivasan Desikan and Gopalaswamy Ramesh, 1st edition, Pearson, 2006.

Reference Book(s):

1. "Effective DevOps: Building a Culture of Collaboration, Affinity, and Tooling" by By Jennifer Davis, Ryn Daniels, O' Reilly Publications, 2018
2. Software engineering – principles and practice Hans Van Vliet, 3rd edition
3. "Software Engineering", International Computer Science Series, Ian Somerville, Pearson Education, 9th Edition, 2009.
4. "Foundations of Software Testing ", Aditya Mathur, Pearson, 2008
5. "Software Testing, A Craftsman's Approach ", Paul C. Jorgensen, Auerbach, 2008.

IEEE SWEBOK, PMBOK, BABOK and Other Sources from Internet.

Software Engineering

Software Metrics

Introduction

"What cannot be measured cannot be Managed"

What does it mean?

without tracking progress and outcomes through data, it's impossible to make informed decisions or control the direction of a project, business, or even personal goals.

It emphasizes the importance of metrics for setting benchmarks, assessing performance, and making necessary adjustments to improve results.

What are Software Metrics in Software Engineering?

Software metrics in software engineering are the standards for estimating the **quality**, **progress**, and **health** of **software development** activity.

A software metric is a quantifiable or countable assessment of the qualities of software.

Characteristics of Software Metrics

Software metrics in software engineering should possess the following characteristics:

- **Quantitative**: Metrics must be quantitative in order to be helpful. It means that metrics can be **stated numerically**.
- **Understandable**: Metric computation should be simple to **grasp**, and the method for **computing** them should be well explained.
- **Applicability**: Metrics should be applied during the early stages of **software development**.
- **Repeatable**: When measured regularly and **consistently in nature**, the metric values should be the same.
- **Economical**: Metric computation should be **cost-effective**.
- **Language agnostic**: Metrics should be **language-independent**, meaning their computation should not depend on any **programming language**.

Software Metrics

IEEE 1061 definition:

“A software quality metric is a function whose inputs are software data and whose output is a single numerical value that can be interpreted as the degree to which the software possesses a given attribute that affects its quality.”

Software Test Related Measurements and Metrics

Software Test Metric: Quantitative Measure of testing process indicating progress, quality, productivity and degree to which system possesses a given attribute

Goal

- Improve efficiency and effectiveness of software testing process
- Make better decisions for future testing process using reliable data

Measurements: critical in performing SQA and testing

Support increasing effectiveness of testing process

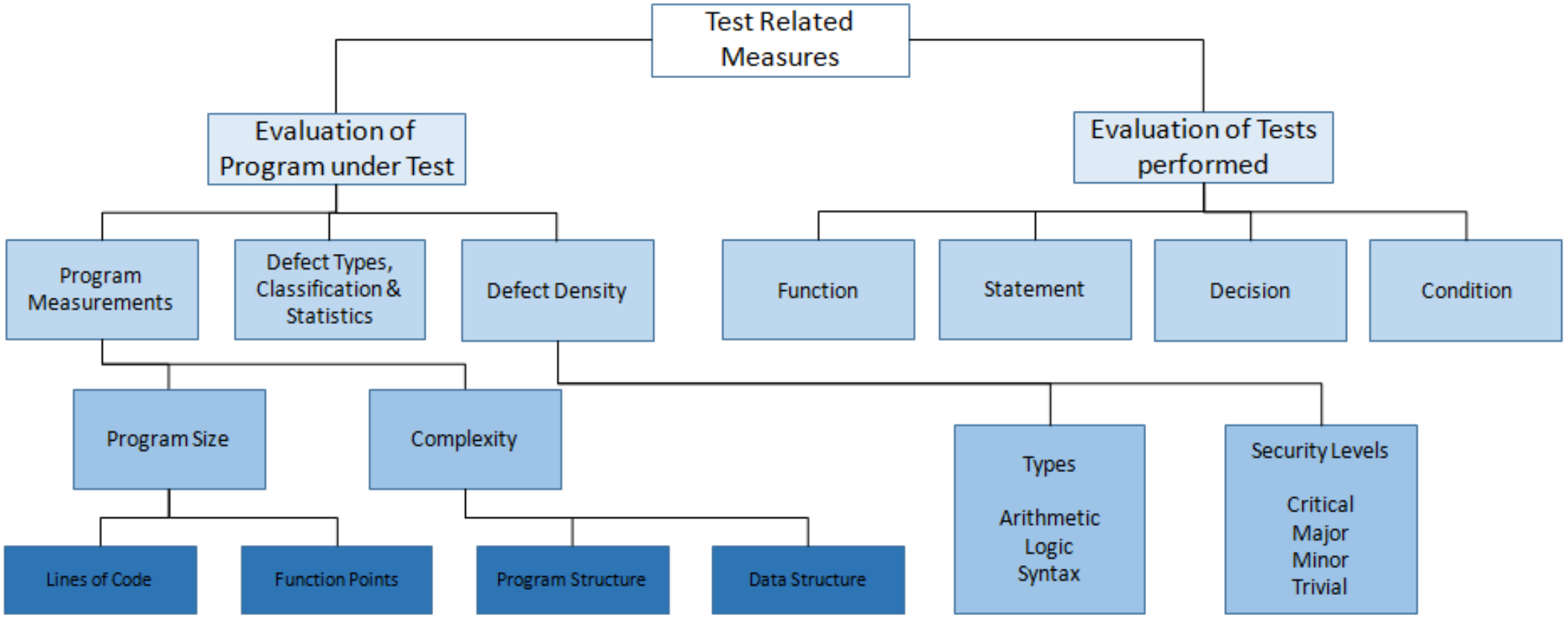
- when used as part of project planning and monitoring
- evaluate the quality of product under test
- helps in defect analysis

Measures can

- represent the quality
- represent test coverage

Software Engineering

Software Test Measurements



Software Test Measurements

Note:

Function Points

Function points measure the size of an application system based on the functional view of the system. The size is determined by counting the number of inputs, outputs, queries, internal files and external files in the system and adjusting that total for the functional complexity of the system.

Function point analysis, originally developed at IBM, has as an advantage its focus on measuring software produced in terms of functionality delivered to the end user, rather than in terms of development deliverables, which have no direct bearing on the end user.

Function Points …

FP Attribute Types or Information Domain Characteristics.

Measurements Parameters	Examples
1.Number of External Inputs(EI)	Input screen and tables
2. Number of External Output (EO)	Output screens and reports
3. Number of external inquiries (EQ)	Prompts and interrupts.
4. Number of internal files (ILF)	Databases and directories
5. Number of external interfaces (EIF)	Shared databases and shared routines.

Function Points …

Weights of 5-FP Attributes

Measurement Parameter	Low	Average	High
1. Number of external inputs (EI)	7	10	15
2. Number of external outputs (EO)	5	7	10
3. Number of external inquiries (EQ)	3	4	6
4. Number of internal files (ILF)	4	5	7
5. Number of external interfaces (EIF)	3	4	6

Risks and challenges associated with metrics

The Streetlight effect

- A known observational bias.
- People tend to look for something only where it's easiest to do so.
- If you drop your keys at night, you'll tend to look for it under streetlights.

Bad statistics

Making inferences

- Provide a theory (from domain knowledge, independent of data)
- Show correlation
- Demonstrate ability to predict new cases (replicate/validate)

Spurious correlations

Confounding variables

Risks and challenges associated with metrics

Spurious correlations

A spurious correlation occurs when two variables appear to be statistically related but have no direct causal connection.

A strong correlation might arise from pure chance, a third confounding factor, or a misinterpretation of complex data. For software development, this can mean mistaking a side effect for a primary cause, which can lead to ineffective or even detrimental changes to development processes.

Example1

Lines of code and bug count: You might observe a strong correlation between the number of lines of code (LOC) and the number of bugs in a software module. A novice might conclude that fewer lines of code will directly cause fewer bugs.

The confounding factor: The actual cause is often the complexity of the module. A more complex feature requires more code, which in turn introduces more opportunities for bugs. Simply writing less code does not reduce bugs; reducing complexity does.

Risks and challenges associated with metrics

Example 2

Number of features and revenue growth: An optimistic product leader might observe that as their team releases more features, the company's revenue also increases. This could lead to a strategy of accelerating feature development to increase sales.

The confounding factor: The company's overall success—driven by market demand, effective marketing, and a strong sales team—is the true cause of the revenue growth. The high feature output and revenue growth are both results of a well-performing, well-funded business. Simply adding more features may even make the product more complex and harder for new users to adopt, potentially hurting growth in the long run.

Example 3

Test coverage and bug reports: A team might notice a high correlation between increased test coverage and a rise in new bug reports. This could lead to the incorrect conclusion that adding more tests actually creates more problems.

The confounding factor: The real cause is a third variable, such as a recent shift to a more aggressive testing strategy. The new strategy uncovers many bugs that were already in the code but were previously undetected, which is the actual purpose of testing.

Risks and challenges associated with metrics

Measurement reliability

Metrics and Incentives

Goodhart's law: "When a measure becomes a target, it ceases to be a good measure."

Incentivizing productivity

What happens when developer bonuses are based on

- Lines of code per day?
- Amount of documentation written?
- Low number of reported bugs in their code?
- Low number of open bugs in their code?
- High number of fixed bugs?
- Accuracy of time estimates?

Software Engineering

Metrics – what you need to know



Metrics are important in Software Engineering



Apply goal-oriented approaches to software metrics



Provide examples of metrics for software qualities and process



Understand limitations and dangers of decisions and incentives based on measurements

Maintenance - Introduction

Software delivered to customers will and can go through a large number of modifications and enhancements due to (??)

1. Fixing Defects reported from the field
2. Correcting requirement and design errors
3. Implementing enhancements
4. Improving performance
5. Adapting the system in response to the environment changes such as
    a. Hardware changes
    b. Software changes within programs that it interfaces with
    c. Network/Telecommunication changes
    d. Changes in standards
6. Retiring outdated systems
7. Interfacing with other software

Maintenance - Introduction

Maintenance: Sustaining process of modifying a software system or component after delivery, to correct faults, improve performance or other attributes, or adapt to changed environment

- Totality of activities required to provide cost-effective support to software
- Modification of software product after delivery to correct faults, improve performance or other attributes, or to adapt to modified environment
- Largest portion of the total life cycle as products are used for a long period of time

Why

To ensure software product continues to satisfy customer needs
Commitment to support system, customers have invested in

Who

Maintainer could be an organization or person responsible for carrying out maintenance activities

Software Engineering

Maintenance - Activities expected and Actions which can help

Activities expected

1. Maintaining control over software's day to day functioning
2. Maintaining control over software modifications
3. Perfecting existing functions
4. Preventing software performance degrading to unacceptable levels

Actions which will support the activities

1. Understand the product which needs to be maintained by studying architecture, design, code, test cases and documentation
2. Discussion with architects and developers
3. Adequate and precise documentation, testable code, code management, instrumentation and debug code
4. Look at present and change request at hand to identify how to satisfy task
5. Look at future and analyze consequences of solution

Software Engineering

Maintenance - Costs

Maintenance consumes a major share of software lifecycle financial resources

- Typically, 70% of total effort spent would be on maintenance
- Total cost also follows this trend

Factors affecting maintenance costs

- Application type
- Software novelty
- Software maintenance staff availability
- Software lifespan
- Hardware Characteristics
- Quality of Software Design, Construction, Documentation, and Testing

Maintenance – Key issues



Technical

1. Code and Documentation Quality
2. Limited Understanding
3. Availability of test environment to reproduce problems
4. Impact analysis

Management

1. Staffing and retaining people with right skill levels
2. Alignment with economic objectives as maintenance does not have much RoI
3. Outsourcing issues
4. Protection of Intellectual Property
5. Control over development process and quality control
6. Learning curve for product
7. Scope of Maintenance

Maintenance - Costs



Cost

1. Application type
2. Lifespan of the system
3. Quality of software artifacts
4. Staff availability

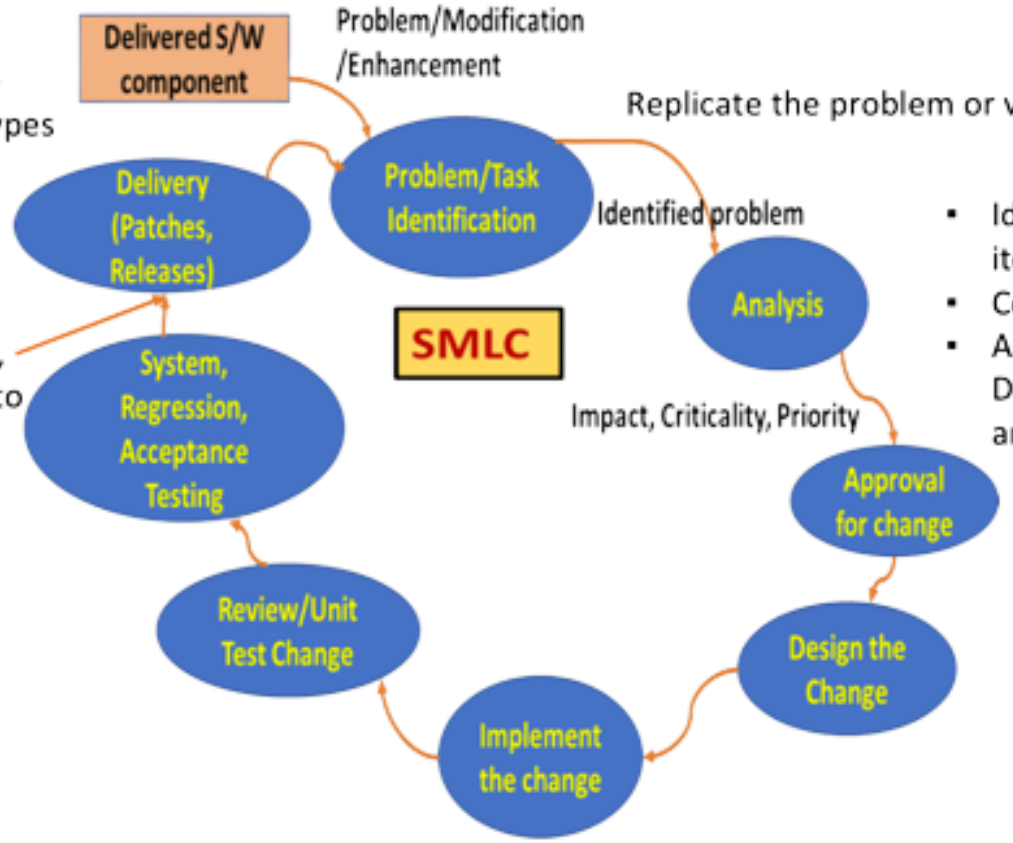
Predictability

1. Measures related to schedule, time required for maintenance, Complexity and Quality of product

Software Engineering

Maintenance – Life Cycle

Different delivery mechanisms with Maintenance lifecycle
Patches – Different Types
Complete products
Application release bundles



Check in the approved, changed components to the CM system

- Identify all affected configuration items
- Consider modification options
- Analyse the impact of problem

Document the request, the analysis, and the suggested options

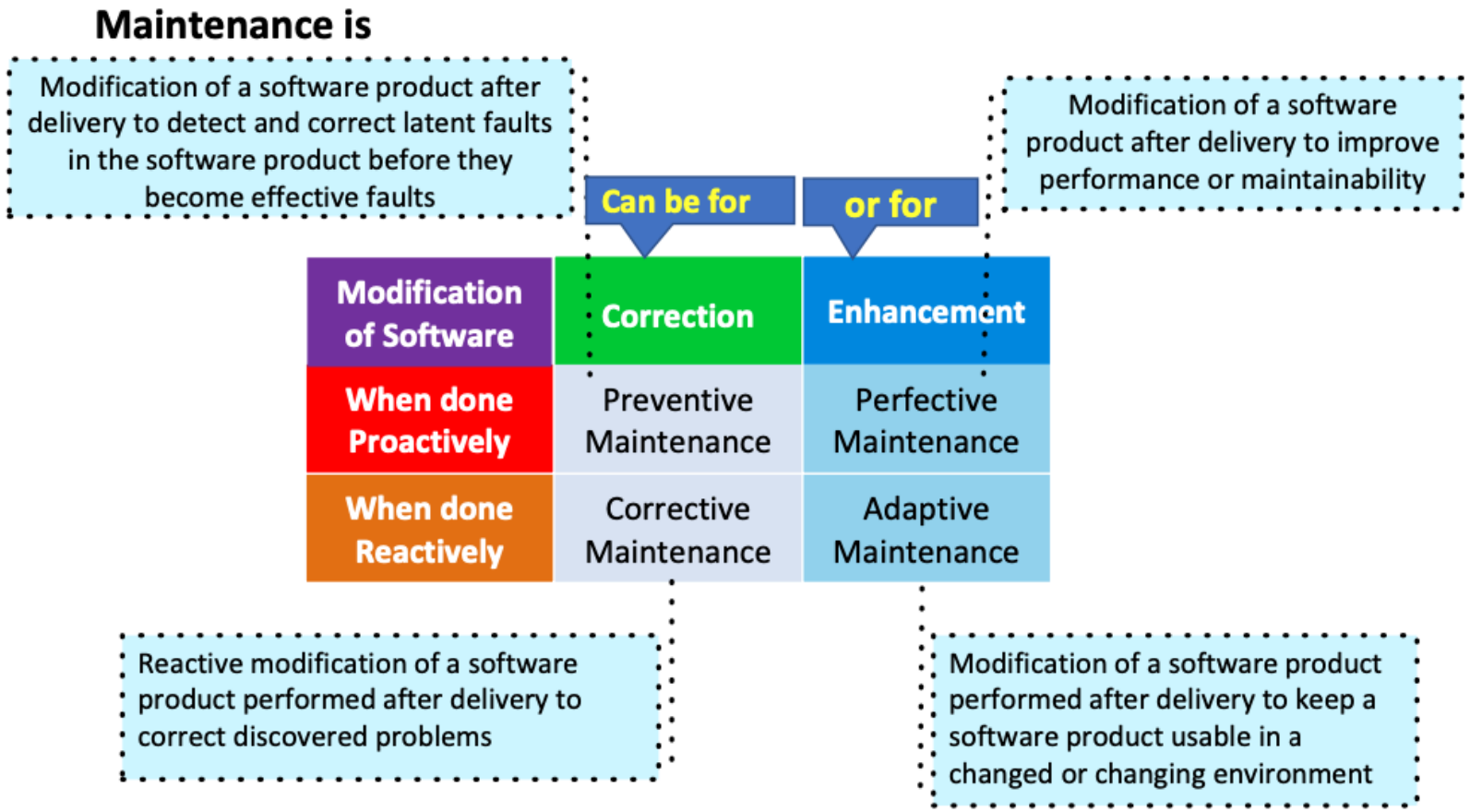
Software Engineering

Maintenance - Tips

- Reproduce the problem.... Before you start fixing it
- Understand the stack traces
- Check your environment variables
- Write a test case that reproduces the problem .. Even better ..Convert the problem into an automated test. ...
- Know your error codes
- Don't assume things work the way they're meant to. ...
- Be clear in your mind about correct behavior. ...
- Pair program whenever possible .. Two pairs of eyes helps
- Guess and check
- Get your code to help you. ...
  - Print
  - Use logs
  - Comment out code and check
- Learn your debugger's capabilities
- Take breaks to get your mind off and restart
- Fix one problem at a time. ...
- Test, Test and Test
- Consider the bigger picture

Software Engineering

Categories of maintenance



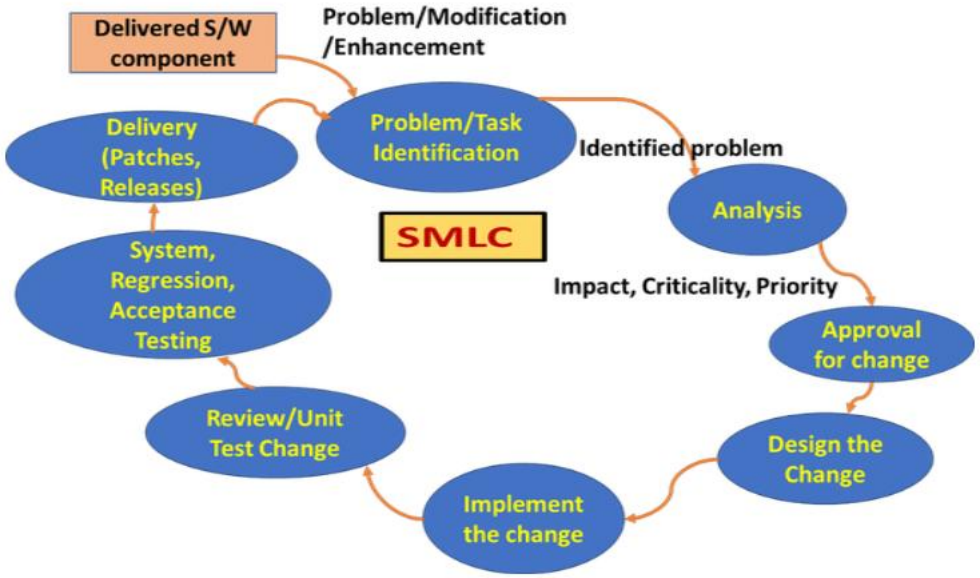
Software Engineering

Maintenance process activities

Implementation of Processes

1. Develop, Document and execute maintenance process plans including usage of tools as part of the process
2. Procedures for handling user reports and modification requests, tracking of problems, responding to users

Follow the Software Maintenance Life Cycle



Maintenance process activities

Migration

1. Develop a migration plan for the fixed/new component and execute the plan
2. This could be
    1. Notifying and replacing the product and restarting
    2. Applying the patch with restart/no-restart based on the product and patch
    3. For a complex product, elaborate set of activities
    4. Migration, verification of migration, **support for older data**

Software Retirement

1. Develop a retirement plan for system including timeline, communicate and execute that plan
2. Retire older version and running of new in parallel
3. Dispose superseded hardware and software

Techniques or Approaches for Maintenance – Reverse Engineering

Reverse Engineering:

passive technique to understand a piece of software prior to re-engineering

- Process of recovering specification and design information about the system from source code

Over time, application may have been corrected, adapted and enhanced

- Application becomes complex, unstable and has unexpected and serious side effects

Identifies the components of a software product and the interrelationships between the components

Creates a representation of the software in a different format than existing code/documentation

Does not modify the product or result in a new product

Typical products of this process are graphs and control flow graphs

Techniques or Approaches for Maintenance – Re-Engineering

Re-Engineering:

process of modifying the software to make it easier to understand, change and extend

- Improving maintainability of system at reasonable cost

Considered from maintenance perspective, when software product is no longer viable to employ standard maintenance techniques to support with reasonable efficiency and productivity

May involve Refactoring

Re-Engineered code is expected to result in reduction of CPU Utilization, Readability, Usability and Performance

Steps

1. Identify current process
2. Document planned reconstruction
3. Reverse engineer
4. Refactor
5. Reconstruct code and data

Open Source Software

Story of Open Source
https://www.youtube.com/watch?v=pVuul3zWj3g

Advantages	Disadvantages
Transparency, gain user trust	Reveal implementation secrets
Many eyes: crowd-source bug reports and fixes	Many eyes: users can find faults more easily
Security: more likely for vulnerabilities to be quickly identified	Security: more likely for others to find vulnerabilities first
Community and adoption: get others to contribute features, build stuff around you, or fork your project	Control: You may not be able to influence the long-term direction of your platnform

Software Engineering

The Cathedral and The Bazaar

Software Engineering Methods
https://www.youtube.com/watch?v=QyvAzL88FgY

Software Engineering

Open Source Licensing

Open Source License types
https://www.youtube.com/watch?v=nFU8KoSgEmk

OSS Stakeholders / Contributors

Core members

- Often (but not always) includes the original creators
- Direct push access to main repository
- May be further split into admin roles and developers

External contributors

- File bug reports and report other issues
- Contribute code and documentation via pull requests

Other supporters

- Beta testers (users)
- Sponsors (financial or platform)
- Steering committees or public commenters (for standards and RFCs)

Spin-offs

- Maintainers of forks of the original repository

Contributing processes

Mature OSS projects often have strict contribution guidelines

- Look for CONTRIBUTING.md or similar

Common requirements:

- Coding style (recall: linters) and passing static checks
- Inclusion of test cases with new code
- Minimum number of code reviews from core developers
- Standards for documentation
- Contributing licensing agreements.

Governance



Some OSS projects are managed by for-profit firms

- Examples: Chromium (Google), Moby (Docker), Ubuntu (Canonical), TensorFlow (Google), PyTorch (Meta), Java (Oracle)
- Contributors may be a mix of employees and community volunteers
- Corporations often fund platforms (websites, test servers, deployments, repository hosting, etc.)
- Corporations usually control long-term vision and feature roadmap

Many OSS projects are managed by non-profit foundations or ad-hoc communities

- Examples: Apache Hadoop/Spark/Hbase/Kafka/Tomcat (ASF), Firefox (Mozilla), Python (PSF), NumPy (community)
- Foundations fund project infrastructure via charitable donations
- Long-term vision often developed via a collaborative process (e.g., Apache) or by benevolent dictators (e.g., Python, Linux)

Corporations still heavily rely on community-owned OSS projects

- Many OSS non-profits are funded by Big Tech (e.g., Mozilla by Google)

Use of Open Source software within companies

Is the license compatible with our intended use?

How will we handle versioning and updates?

- Does every internal project declare its own versioned dependency or do we all agree on using one fixed (e.g., latest) version?
- Sometimes resolved by assigning internal “owners” of a third-party dependency, who are responsible for testing updates and declaring allowable versions.

How to handle customization of the OSS software?

- Internal forks are useful but hard to sync with upstream changes.
- One option: Assign an internal owner who keeps internal fork up-to-date with upstream.
- Another option: Contribute all customizations back to upstream to maintain clean dependencies.

Security risks? Supply chain attacks on the rise.

Software Patents

What is a patent?

“A patent is an exclusive right granted for an invention, which is a product or a process that provides, in general, a new way of doing something, or offers a new technical solution to a problem. To get a patent, technical information about the invention must be disclosed to the public in a patent application.”

WIPO
WORLD
INTELLECTUAL PROPERTY
ORGANIZATION

What rights do patents grant?

Patents don’t give you the right to make, use, or sell an invention.

Patents do give you the right to exclude others from making, using, and selling an invention for the term of a patent (20 years)

- stop or sue others
- licensing and royalties

Copyright v/s Patent

Copyrights cover the details of expression of a work
Copyrights don't cover any ideas

- Patents only cover ideas and the use of ideas

Copyrights happen automatically.

Patents are issued by a patent office in response to an application.

Software Engineering

Patent or Not ?

- Running bingo on a computer
- Using a computer to help users plan meals while achieving diet goals
- Using a computer to order a pizza with customized toppings
- Prompting a user before establishing a new network connection
- Automatically notifying users when an item is picked up or delivered
- Using a computer network to ask people to complete tasks and then wait for them to do them
- Using SMS to perform tasks (e.g., checking bank balance)
- Selecting ALL images in a CAPTCHA that match a given text

Network attacks ..

- Examples
  - Person-in-the-middle attack
  - Sniffing
  - Spoofing
- We must assume the network is not secure
- We must guard against a compromised network

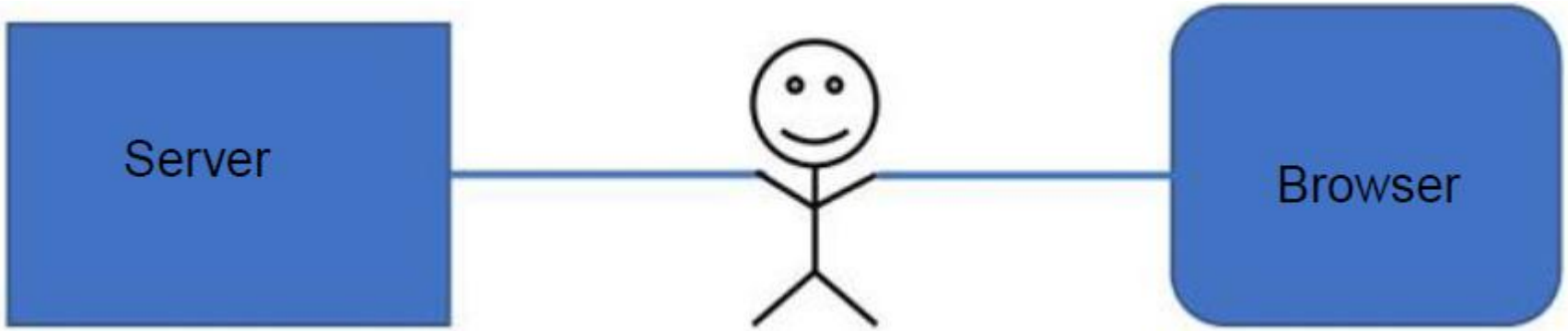
Software Engineering

Person-in-the-middle

Someone that can intercept network traffic

Can read the messages (coming and going)

Can change the messages before sending them on (to the correct or incorrect destination)



Software Engineering

Sniffing, Eavesdropping, etc

- You can listen to the traffic going by on the net.
- This is typically traffic on your subnet
  - Still, it can be interesting.
  - If you can plug in to the backbone…



Spoofing

- Pretending to be someone you're not
- IP spoofing
- Pretending to a "client" you're not (with a specific IP address)
- E-mail Spoofing
- DNS spoofing
  - Pretending to be a server that you're not
  - Fool a DNS server to give out incorrect IP addresses for DNS Names
  - Note: also, be careful of typos or similar characters attacks:

Software Engineering

Big 3 concepts of network security

- Authentication
- Authorization
- Confidentiality

Authentication: Authentication is knowing with whom you are communicating. User knowing the server and/or server knowing who is the user.

Which is more important ?

Big 3 concepts of network security

Authorization: User having privilege to perform an operation on server.

Confidentiality:

- Communicating without others knowing what's said.
- Intermediaries cannot change what was said.
- Typically includes protection from replay attack.
- (But typically, does not provide secrecy of communication. Others can know that communication occurred.)

Which of the Big 3 protect you from?

Which of the “big-three” protect you from:

- **Sniffing** ? Confidentiality
- **Spoofing** ? Authentication
- **Person-in-the-middle Attack** ? Authentication + Confidentiality

Software Engineering

Concept of privacy

Moral Right of individuals to be left alone, free from surveillance or interference from other individuals or organizations, including state.

How is Privacy protected?

- Laws, self-regulation, technology
- Notice and access control over collection, use, deletion, sharing
- Collection limitation
- Use limitation
- Security and accountability

US FTC's Fair Information Practice Principles

FTC – Federal Trade Commission

Notice/Awareness (Core Principle)

a. Disclose practices.

Choice/ Consent (Core Principle)

a. Opt-in, opt-out

Access/ Participation

a. Users should be able to review & correct their information.

Integrity/ Security

a. Ensure secure, Limited Access

Enforcement

a. Mechanism for handling violations

Why SWEs should care about privacy

- Ethical Questions
- Laws restricting data collection by government and agencies
  - Freedom of information Act (1966), Privacy Act (1974), Electronics and communication act (1986)…
- Laws restricting data collection in different economic sectors
  - COPPA (**Children's Online Privacy Protection Act)**, HIPPA (**Health Insurance Portability and Accountability Act**), FERPA (**Family Educational Rights and Privacy Act**) etc.
- State Laws (eg: CCPA - **Central Consumer Protection Authority**) and local laws.
- EU –General Data Protection Regulation

Goals of privacy engineering

- Ensuring legal compliance
  - GDPR(General Data Protection Regulation), etc.
- Aligning with consumer expectations
  - Transparency about data practices.
  - Accurate statements about privacy policies.
- Building trust and goodwill.
  - Commitment to protecting user data.
- Competing on privacy protection.
  - Privacy as a core value.
- Promoting privacy as a societal value.
  - Safeguarding privacy rights and advocating for ethical data practices.

Software Engineering

Mechanisms in privacy engineering

- Selective Data Collection
  - Purpose-driven, minimize amount of personal information.
- Data Minimization
  - De-identification, pseudonymization, anonymization
  - Remove sensitive data
- Data Retention Policies
- Cryptographic tools
  - Confidentiality
- Access controlled and secure data storage
- Socio-technical processes and audits
- Threat modelling
- Privacy-by-Design (PbD)

Threat Modelling

What is Privacy Threat Modeling

Privacy threat modeling is an activity to identify potential privacy threats in an application or system. By performing privacy threat modeling, it is possible to identify early on in a development process potential risks to privacy for various stakeholders.

Privacy threat modeling will identify privacy threats so that countermeasures can be developed before an application or system is in use.

- Applicable to both security and privacy.
- A wide variety of possible security and privacy threats.
  - How can we organize our analysis ?
- Basic Idea : Systemic methodology to identify possible threats and methods to mitigate these threats.
- Approach: Use a taxonomy of possible threats.

Software Engineering

LINDDUN Taxonomy of privacy threats



More information at: https://threat-modeling.com/linddun-threat-modeling/

Software Engineering

Code and Branch Coverage



Code coverage metrics

Code Coverage testing: Examines what fraction of the code under test is reached by existing unit tests.

Structural code coverage metrics include :

- Statement coverage
- Condition coverage
- Decision coverage

Which type of coverage requires the most tests?

Software Engineering

Code coverage: The basics

Average of the absolute values of an array of doubles

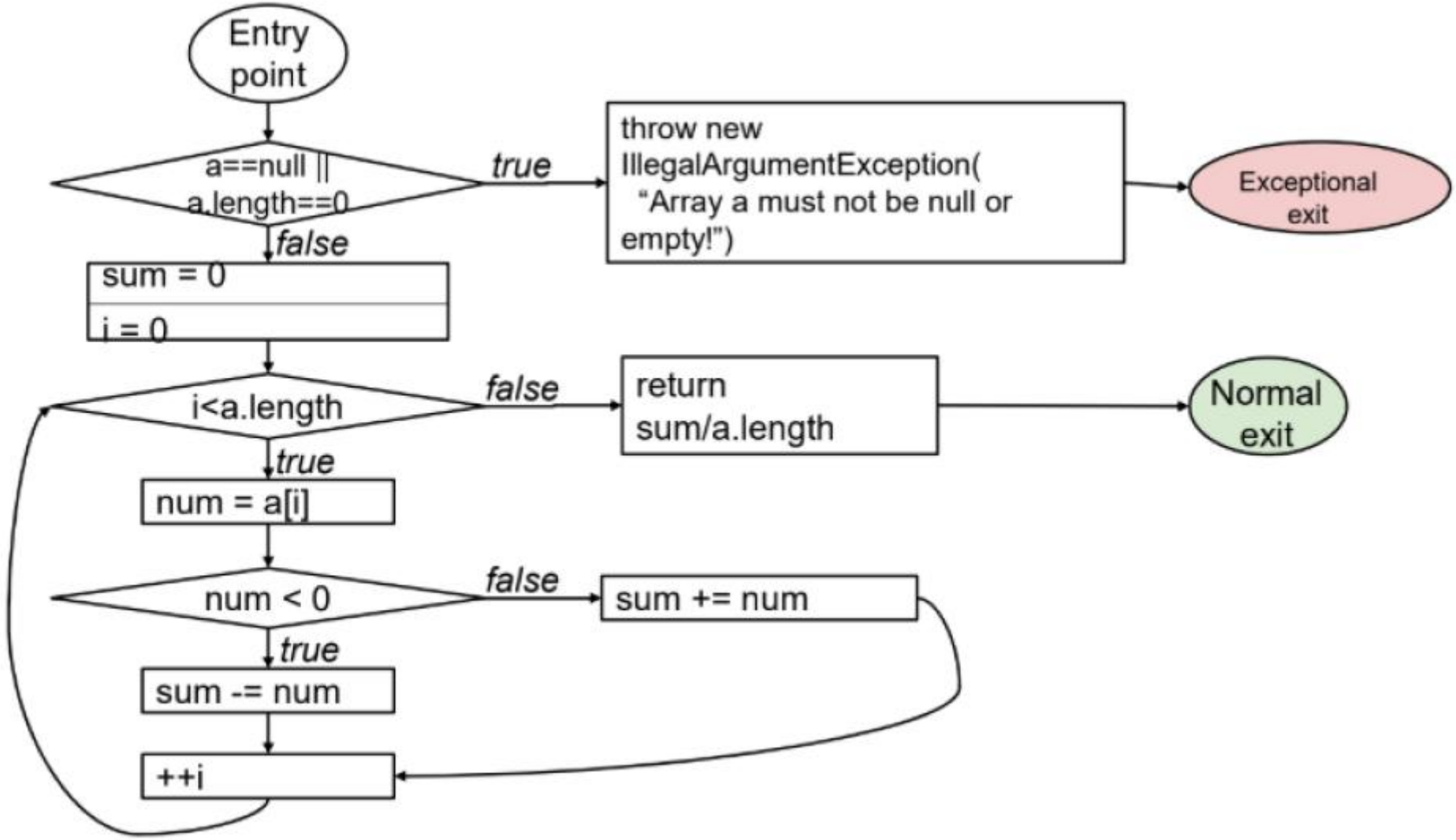
```
public double avgAbs(double ... numbers) {

  // We expect the array to be non-null and non-empty
  if (numbers == null || numbers.length == 0) {
    throw new IllegalArgumentException("Nums cannot be null or empty!");
  }

  double sum = 0;
  for (int i=0; i<numbers.length; ++i) {
    double d = numbers[i];
    if (d < 0) {
      sum -= d;
    } else {
      sum += d;
    }
  }
  return sum/numbers.length;
}
```

Software Engineering

Create the control flow graphs



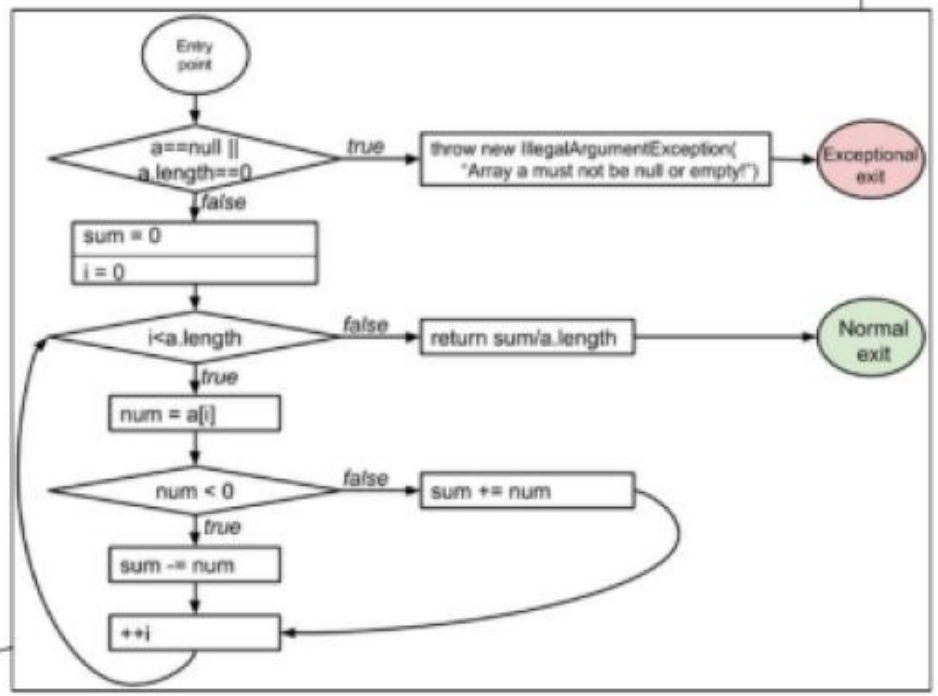
Software Engineering

Align the two to help identify tests

```
public double avgAbs(double ... numbers) {

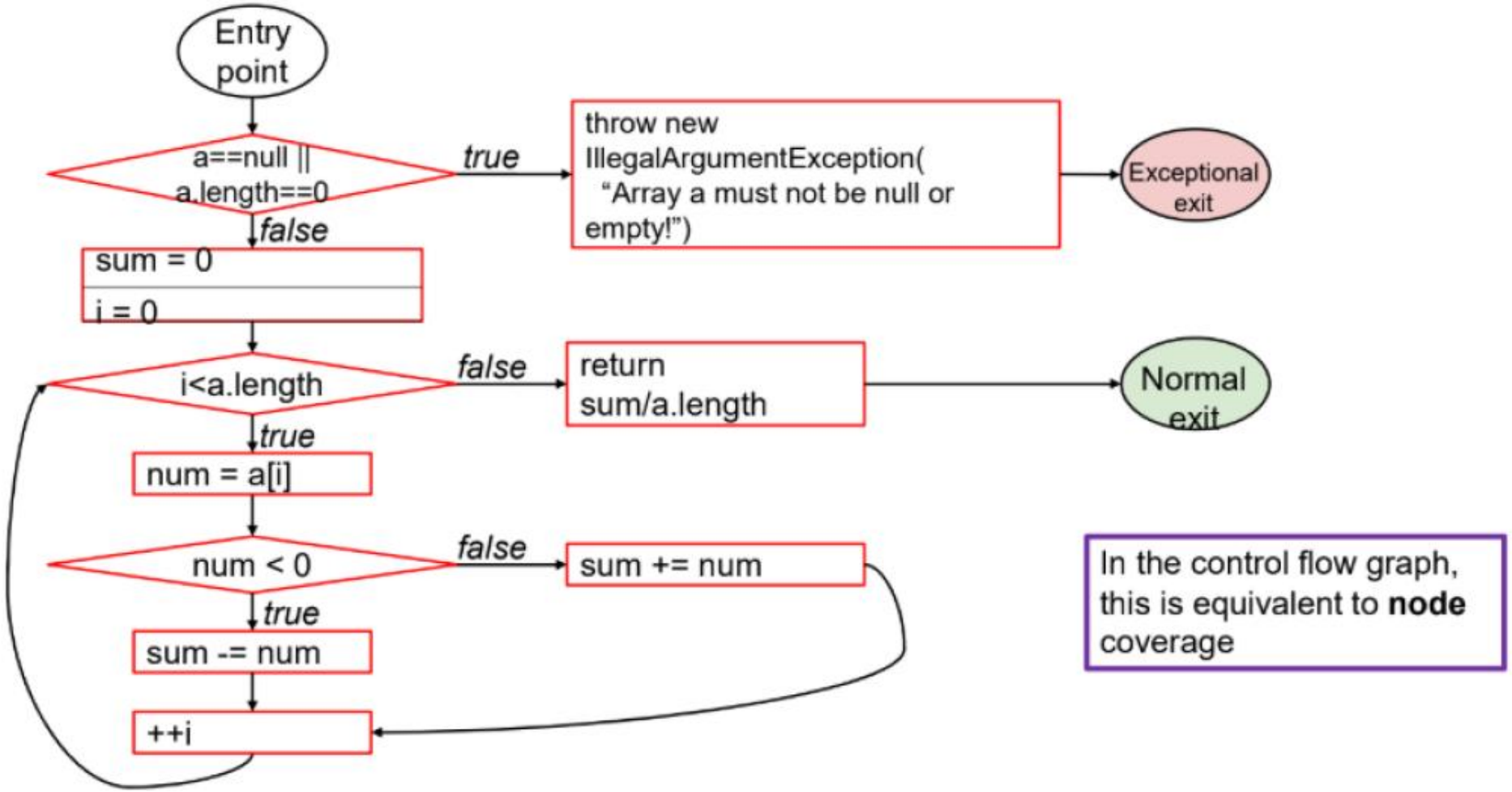
  // We expect the array to be non-null and non-empty
  if (numbers == null || numbers.length == 0) {
    throw new IllegalArgumentException("Numbers must not be null or empty!");
  }

  double sum = 0;
  for (int i=0; i<numbers.length; ++i) {
    double d = numbers[i];
    if (d < 0) {
      sum -= d;
    } else {
      sum += d;
    }
  }
  return sum/numbers.length;
}
```



Software Engineering

Statement coverage



In the control flow graph, this is equivalent to **node** coverage

Every statement should be executed at least once by the tests

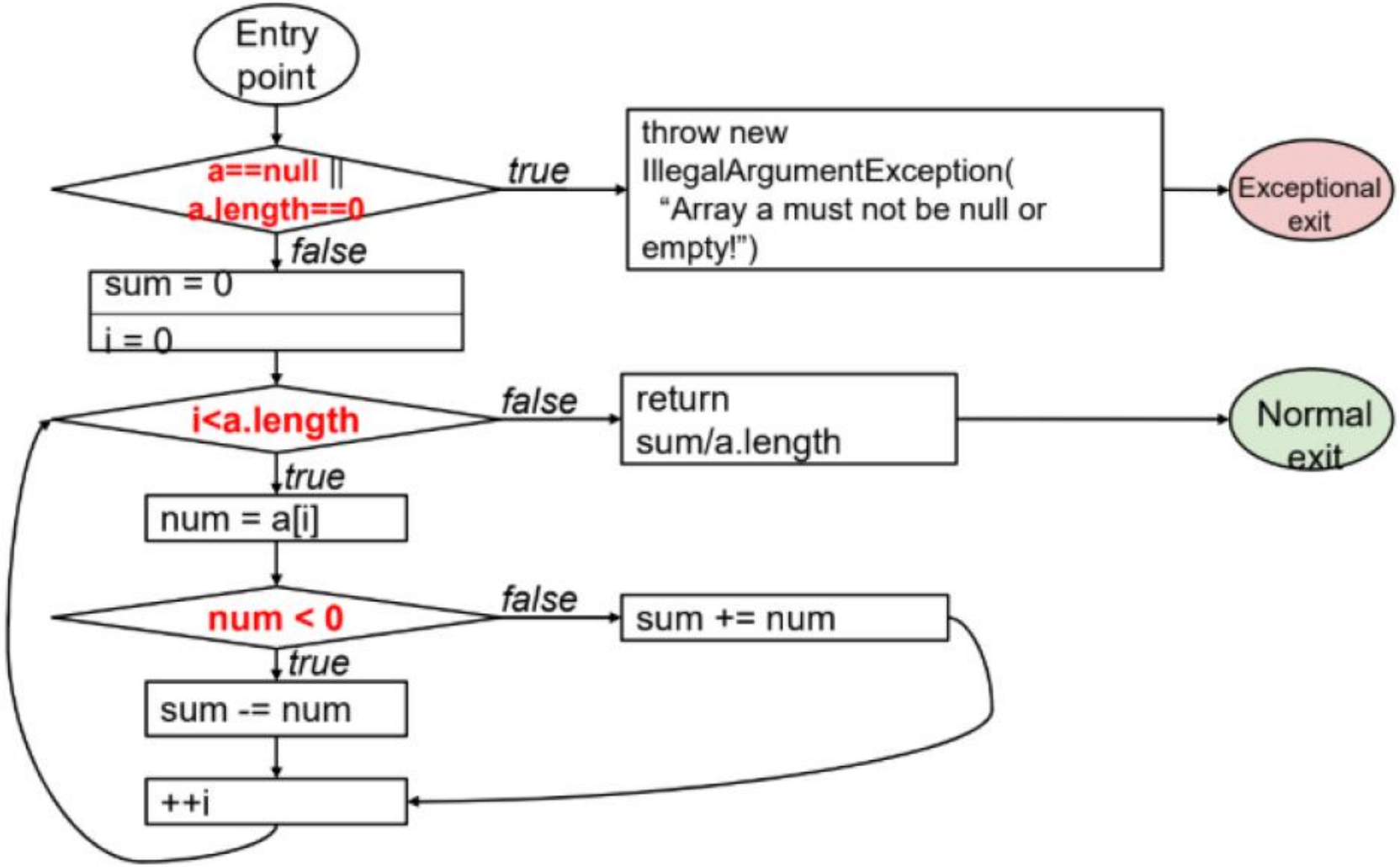
Condition coverage

Condition
a Boolean expression that cannot be decomposed into simpler Boolean expressions (atomic)

Condition coverage
every condition in the program must take on all possible outcomes (true/false) at least once.

Software Engineering

Condition coverage



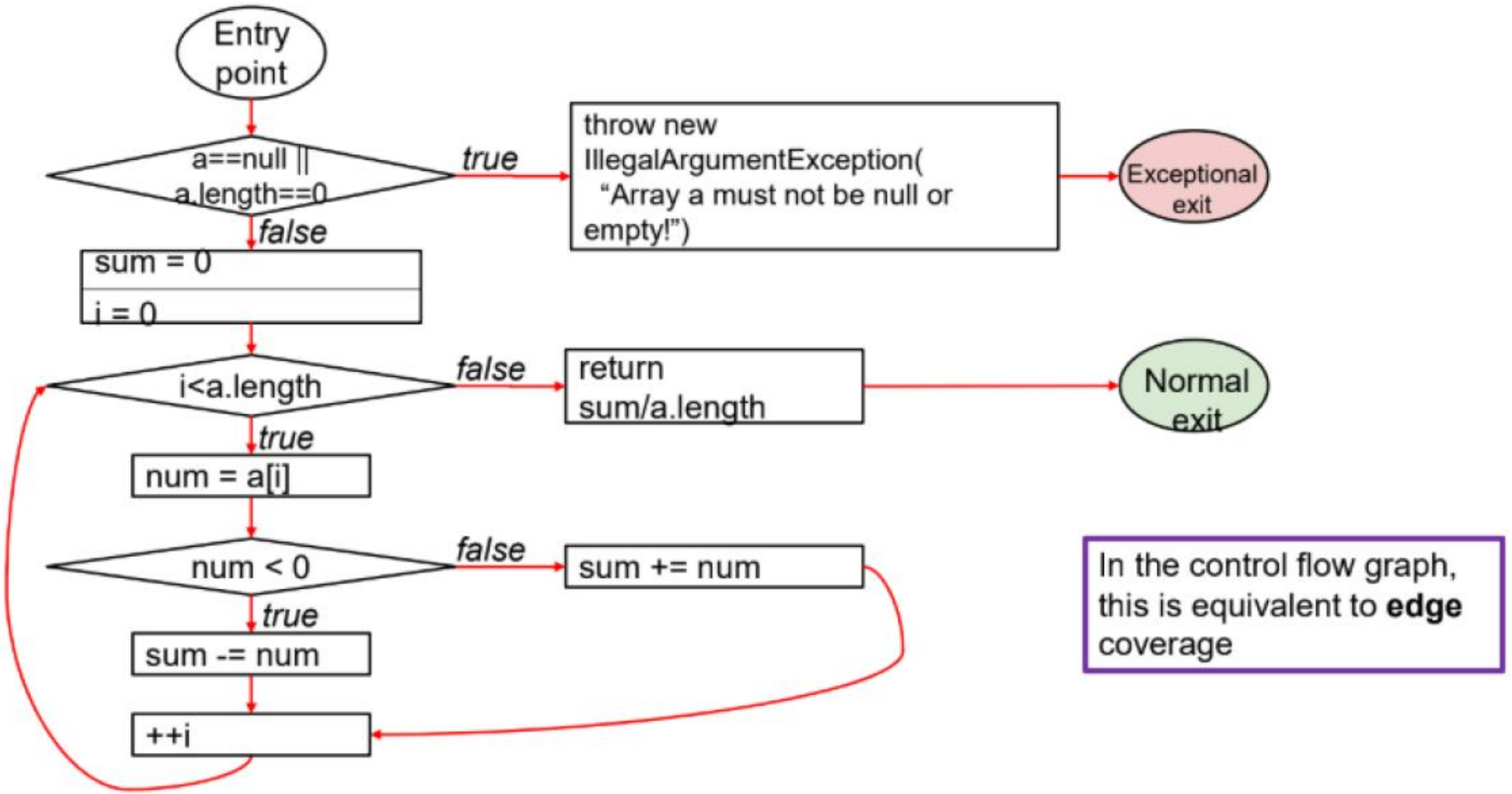
Decision coverage

Decision: a Boolean expression that is composed of conditions, using zero or more logical connectors.

Decision coverage: every decision in the program must take on all possible outcomes (true/false) at least once.

Software Engineering

Decision coverage



In the control flow graph, this is equivalent to **edge** coverage

Coverage

How much coverage is enough – 100 %

Good resource on code coverage and code coverage tools:
https://www.atlassian.com/continuous-delivery/software-testing/code-coverage

And a good list of coverage tools:
https://www.browserstack.com/guide/code-coverage-tools

Code coverage takeaways

- Code coverage can provide valuable insights into your code and into your testing adequacy
- It is intuitive to interpret
- There are great tools available to help compute code coverage of your tests
- Code coverage itself is not sufficient to ensure correctness
- Code coverage is well known and used in industry

Software Engineering

Ethics in Software

Ethics, also called moral philosophy is the discipline concerned with what is morally good and bad and morally right and wrong.

As computer scientists and software engineers you will be faced with making difficult decisions.

•Personal ethics (Integrity, Objectivity, Professional Competence, Confidentiality)

•Professional ethics

- Reuse of source code
- Reuse of copyrighted materials
- Enforcing strength of passwords
- Knowledge of flaw or bug
- Design of potentially dangerous software feature
- Discovery of security flaw

Often, there are clear cut laws to dictate the decision

Ethics

Personal Ethics

- Values or viewpoints an individual uses to influence and guide his or her personal, social, or professional behavior
- Values are reinforced and strengthened from childhood

Business Ethics

- Values or viewpoints which a business and “individuals when doing business” use to influence and guide behavior
- Focus on behaviors pertaining to work

Professional Ethics

- Principles that govern the behavior and judgment while performing his/her profession.

Ethics

Public: software engineers act consistently with public interest
Client and Employer: Act in a manner that is in best interests of their client or employer and consistent with public interest
Product: Products and Related modifications meet highest professional standards
Judgement: maintain integrity and independence in professional judgment
Management: subscribe to and promote an ethical approach to management
Profession: advance the integrity and reputation of the profession consistent with public interest
Colleagues: fair and supportive of their colleagues
Self: lifelong learning and promote ethical approach

Software Engineering

Code of Ethics

As an ACM member I will ....

- Contribute to society and human well-being.
- Avoid harm to others.
- Be honest and trustworthy.
- Be fair and take action not to discriminate.
- Honor property rights including copyrights and patent.
- Give proper credit for intellectual property.
- Respect the privacy of others.
- Honor confidentiality.

More on testing …



Integration testing

Two ways to go about

Big bang approach and Incremental integration

Big bang approach

Design, code, test, debug each class/unit/sub-system separately
Combine them all
Hope for the best.

More on testing …



Incremental integration

- Incrementally from the top, bottom or “sandwich” ?
- “Sandwich” integration by fleshing out a skeleton system.
- Connect top-level UI with crucial bottom-level components.
- Add middle layers incrementally.
- More common and agile approach.

More on testing …

Different ways to approach integration

Top Down Integration :

- Start with outer UI layers and work inward.
- Must write (lots of) lower level stubs for UI to interact with.
- Allows postponing tough design/implementation decisions (bad ?).

Bottom up Integration :

- Start with lower level data/logic layers and work outwards.
- Must write upper level stubs to drive these layers.
- Won't discover high-level/ UI design flaws until late.

More on testing …

Which approach to adopt?

- If you have UI requirements clarified → Top-down makes sense
- If database schema is critical → Bottom-up
- Most agile projects → Sandwich (build skeleton, iterate)

More on testing …

Regression testing

- Regression testing is testing the system to check that changes have not ‘broken’ previously working code.
- In a manual testing process, regression testing is expensive but, with automated testing, it is simple and straightforward. All tests are rerun every time a change is made to the program.
- Tests must run ‘successfully’ before the change is committed.

More on testing …

Release testing and system testing

Release testing is a form of system testing.

Important differences:

- A separate team that has not been involved in the system development, should be responsible for release testing.
- System testing by the development team should focus on discovering bugs in the system (defect testing). The objective of release testing is to check that the system meets its requirements and is good enough for external use (validation testing).

More on testing …

Performance testing

Part of release testing may involve testing the emergent properties of a system, such as performance and reliability.

Tests should reflect the profile of use of the system.

Performance tests usually involve planning a series of tests where the load is steadily increased until the system performance becomes unacceptable.

Stress testing is a form of performance testing where the system is deliberately overloaded to test its failure behaviour.

More on testing - Penetration testing

Penetration testing

Overview

- An **authorized, simulated cyberattack** on a system to evaluate its security
- Goal: identify and exploit vulnerabilities before real attackers do.
- Complements other testing (e.g. vulnerability scans) by mimicking attacker behavior.
- Findings include weaknesses in networks, applications, configurations, or human factors.
- Penetration testing (“pentesting”) involves security experts using the same tools and techniques as attackers to probe a system. It combines automated scanning with manual exploitation.
- The tester reports discovered vulnerabilities and how they can be exploited, along with recommended fixes.

More on testing – Penetration testing



Types of penetration testing

White-box testing: Tester has full knowledge of target (source code, architecture)

Black-box testing: Tester knows only public information (like an external attacker)

Gray-box testing: Tester has partial knowledge (some credentials or architecture) – a hybrid approach.

Different approaches suit different goals: white-box can be more thorough by using internal insights, while black-box best simulates a real external attacker. Gray-box balances both.

Typical industry products (e.g. HW products) go for white box testing

Software Engineering

More on testing – Penetration testing process

- **Reconnaissance:** Gather information on target (domains, IPs, tech stack) using public sources (WHOIS, Shodan, etc.) to understand potential attack surfaces
- **Scanning:** Probe the target for open ports, services, and known vulnerabilities (e.g. using Nmap or vulnerability scanners)
- **Vulnerability Assessment:** Analyze scan results, identify high-risk issues, and prioritize targets
- **Exploitation:** Attempt to exploit confirmed vulnerabilities (e.g. SQL injection, XSS (Cross site scripting), misconfigurations) to prove impact and support remediation planning
- **Reporting:** Document findings, evidence of exploitation, and remediation advice; often includes screenshots or logs
- A typical pentest follows these stages to systematically uncover security flaws. For example, the Open Web Application Security Project (OWASP) and industry standards describe similar five-phase methodologies. In exploitation, testers might demonstrate how an attacker could gain admin access or steal data, which helps justify fixes.

More on testing …

Benefits of penetration testing

Prevents breaches: Finds critical flaws before attackers do.

Compliance and Assurance: Fulfills requirements (e.g. PCI DSS mandates regular pentests) and boosts stakeholder confidence.
(Payment Card Industry Data Security Standard)

Red Teaming and Risk Assessment: Reveals not just technical bugs but also process issues (e.g. weak passwords, unpatched servers) and helps organizations prioritize security improvements.

Pentesting provides a reality check of an organization's security.

In practice, security audits that include pentesting have helped companies avoid costly data breaches by uncovering critical flaws in time.

More on testing – Penetration testing tools (OSS)

Nmap: Network mapper/scanner to discover hosts, ports, services

Metasploit Framework (Community Edition): Exploitation platform with many payloads (partially open source).

OWASP ZAP: Web application scanner (DAST) for finding common web vulnerabilities

SQLMap: Automates SQL injection discovery and exploitation in web applications

Wireshark: Network protocol analyzer (for packet capture & analysis).

ZAP: widely used web app scanner.

Software Engineering

Fuzzing

Overview

Definition:

Automated testing by feeding a program random, malformed, or unexpected inputs

Purpose:

Trigger crashes, assertion failures, memory errors, or other bugs that might be security vulnerabilities.

Key idea:

Let a *fuzzer* generate many test cases (often continuously) and watch for faults that manual tests miss.

Fuzzing (or “fuzz testing”) is a dynamic analysis technique. In practice, a fuzzer iteratively generates inputs and runs the target program, checking if it crashes or behaves incorrectly

Unlike traditional test suites with fixed cases, fuzzers explore thousands to millions of inputs, including edge cases and random data. This exposes hidden bugs, especially in low-level code (e.g. C/C++ memory errors) or poorly-validated parsers.

Types of fuzzers

Mutation-based fuzzers: Start from valid sample inputs and modify them (flip bits, alter lengths, insert special values)

Generation-based fuzzers: Use a model of the input format (specification or grammar) to generate new inputs from scratch.

White-box fuzzers: Have full knowledge of the target (source code/structure); can use e.g. symbolic execution or instrumentation.

Gray-box fuzzers: Use feedback (e.g. code coverage) but treat the target as a black-box binary with limited instrumentation.

Black-box fuzzers: No knowledge of internals; treat the program purely as an external interface, relying on random inputs only.

Many fuzzers fall on a spectrum of intelligence. For example, a “dumb” fuzzer might choose purely random bytes, while a “smart” fuzzer tries to produce syntactically valid or interesting inputs. In fact, fuzzers can be categorized by how much they know: white-box fuzzers know internal details, black-box know nothing, and gray-box have some instrumentation.

Software Engineering

Fuzzing process

Input Generation: The fuzzer produces test cases (either by mutating seed inputs or generating new data)

Program Execution: Each test input is fed to the target program, which is run under a monitor (often with sanitizers like AddressSanitizer) to detect errors.

Monitoring/Analysis: The fuzzer watches for crashes, hangs, leaks, or assertion failures. It logs inputs that cause such failures.

Feedback-Driven Loop: Coverage-guided fuzzers analyze which new code paths were exercised. Inputs that increase coverage are “favored” and used to generate further mutations

Modern fuzzers operate in three phases: they generate inputs, run the target, and then use the results to drive further testing. For instance, AFL and libFuzzer track execution to find new branches.

Fuzz testing steps (generate or mutate inputs, run the target, monitor for crashes, and analyze failures) are repeated thousands of times to uncover crashes or vulnerabilities.

Fuzzing - value

Uncovers deep bugs: Fuzzing has found countless memory safety and crash bugs in critical software (e.g. browsers, SSL libraries) that were invisible to code review

Broad applicability: It can test anything with a programmable interface: libraries, file parsers, network servers, even entire kernel modules.

Continuous security: Services like Google's OSS-Fuzz continuously fuzz open-source projects and report new CVEs, fixing bugs before users find them.

Complements other testing: Unlike pentesting (which targets high-level behavior), fuzzing excels at low-level code paths. Together they strengthen security validation.

Fuzzing is especially valuable for **security validation** because it simulates random/malicious inputs.

Software Engineering

Fuzzing tools – Open Source

AFL / AFL++ (C/C++): A coverage-guided mutation fuzzer; widely used.

libFuzzer (C/C++): A library-based fuzzer integrated into LLVM/Clang; generates inputs in-process using sanitizers.

Honggfuzz (C): A general-purpose feedback-driven fuzzer supporting sanitizers and code-coverage.

Syzkaller (C, Linux Kernel): Kernel fuzzer by Google, uses system call models to fuzz the Linux kernel.

Radamsa: A gray-box dumb mutational fuzzer (common for file-format fuzzing).

Hypothesis (Python): A property-based testing library – part of the lab later on.

Python fuzzers: e.g. BooFuzz (network protocol fuzzer), python-afl (bindings for AFL), PyFuzzer packages.

There are many open-source fuzzers. For C/C++ code, AFL and libFuzzer are standards. Google's OSS-Fuzz runs large-scale fuzzing on thousands of projects (finding ~10,000 bugs since launch). Python has its own ecosystem.

Software Engineering

Fuzzing – case studies

Mozilla Firefox (2022–2023): Mozilla's fuzzers detected *20 vulnerabilities* (5 critical, 15 high) in Firefox 102/103, such as memory safety bugs, before these versions shipped

OpenSSL (2022): Google's OSS-Fuzz found *two critical CVEs (CVE-2022-3602, CVE-2022-3786)* – stack buffer overflows in the Punycode parser. These were fixed before public exploitation

Linux Kernel: The Syzkaller project has unearthed *thousands of kernel bugs* since 2015, many being privilege escalations (e.g. CVE-2023-0386).

Other: Tools like AFL have been used to fuzz browsers (Chrome/Firefox), PDF readers, media libraries, often yielding dozens of CVEs each year.

These examples (all within 5 years) highlight fuzzing's impact. They show that even mature, critical software still benefits from automated fuzzing. Many companies now integrate fuzzing into their CI pipelines for this reason.

Software Engineering

Stubs

What's a Stub?

- A controllable replacement for a software unit. Used to simulate difficult-to-control elements or components not yet developed

Why you need them:

- Component B doesn't exist yet (you're still building it)
- Component B is hard to control (it's a 3rd-party service, database, network)
- Component B is slow (real database calls take time)
- Component B has side effects (sending emails, charging cards)

Example: Testing registration logic without touching real database:

- Create a StubDatabase that returns fake data
- Registration logic can be tested in isolation
- No risk of data corruption

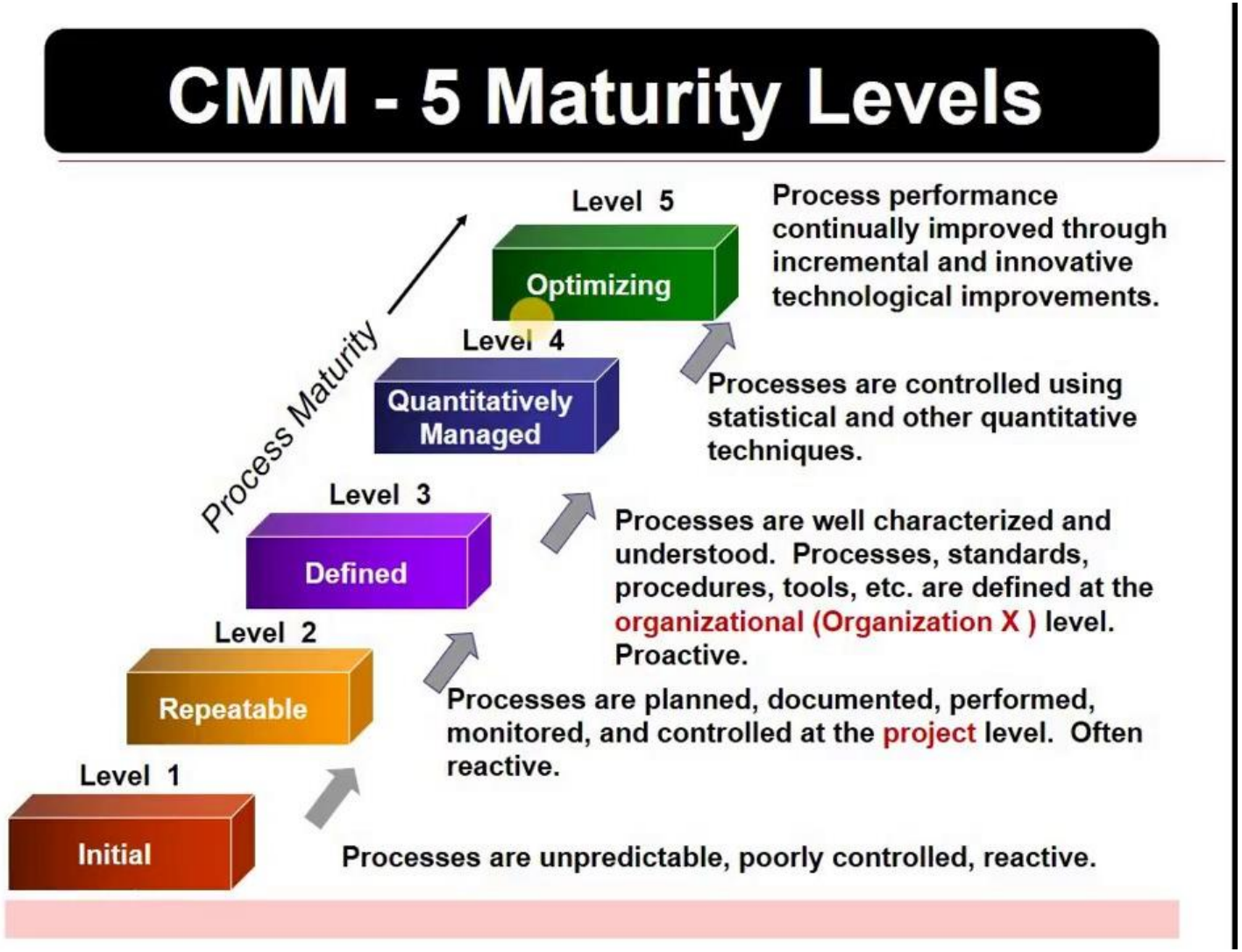
Key point: Minimize code changes when you remove the stub.

Capability Maturity

Capability Maturity Model (CMM)

- Developed by Software Engineering Institute of Carnegie Mellon University
- Tool for objectively assessing the capability of vendor to deliver software
- Maturity model: set of structured levels that decide how well the behaviors, practices and processes of an organization can reliably and sustainably produce outcomes
- Evolutionary improvement path for software organization
- Benchmark for comparison of software development processes and an aid to understanding

Software Engineering Institute – CMM …

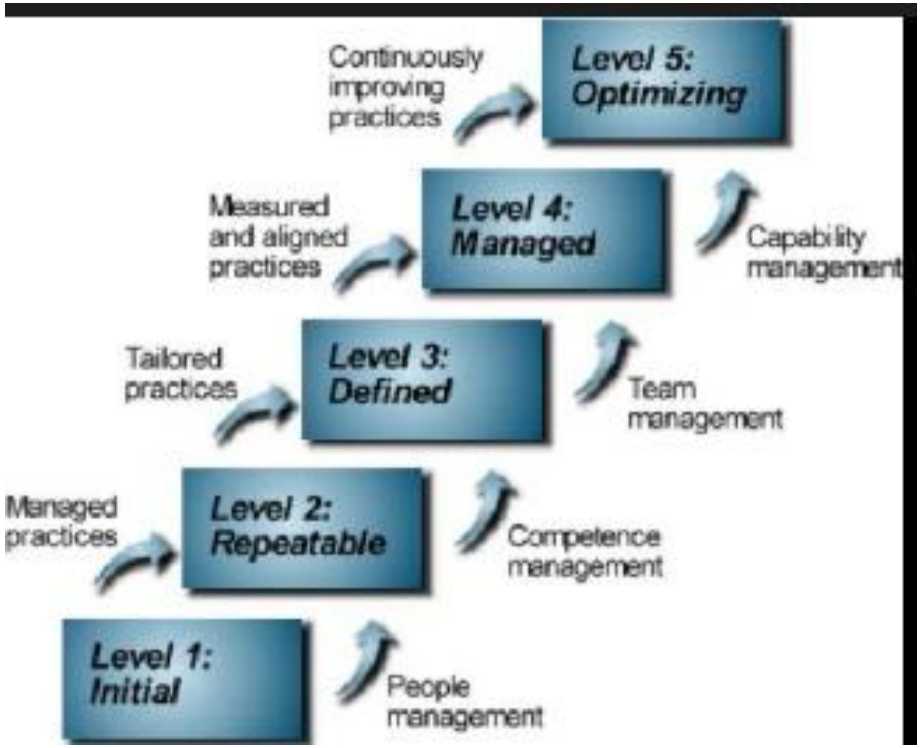


Software Engineering

PCMM

What's PCMM?

The People Capability Maturity Model (P-CMM) is a framework developed by Carnegie Mellon University to help organizations improve their human resource practices and continuously develop their workforce's knowledge, skills, and motivation.





THANK YOU

M S Anand
Department of Computer Science Engineering

anandms@pes.edu